arXiv:2510.09783v2 [cs.LG] 8 Jun 2026

Large Language Models for Imbalanced Classification: Diversity makes the difference

Dang Nguyen[1], Sunil Gupta[1], Kien Do[1], Thin Nguyen[1], Taylor Braund[2], Alexis Whitton[2], Svetha Venkatesh[1]
[1]Applied Artificial Intelligence Initiative (A^2I^2), Deakin University, Geelong, Australia
[2]Black Dog Institute, University of New South Wales, Sydney, Australia
[1]{d.nguyen, sunil.gupta, k.do, thin.nguyen, svetha.venkatesh}@deakin.edu.au
[2]{t.braund, a.whitton}@blackdog.org.au

***Abstract*—Oversampling is one of the most widely used approaches for addressing imbalanced classification. The core idea is to generate additional minority samples to rebalance the dataset. Most existing methods, such as SMOTE, require converting categorical variables into numerical vectors, which often leads to information loss. Recently, large language model (LLM)–based methods have been introduced to overcome this limitation. However, current LLM-based approaches typically generate minority samples with *limited diversity*, reducing robustness and generalizability in downstream classification tasks.**

To address this gap, we propose a novel LLM-based oversampling method designed to enhance diversity. First, we introduce a sampling strategy that conditions synthetic sample generation on both minority labels and features. Second, we develop a new permutation strategy for fine-tuning pre-trained LLMs. Third, we fine-tune the LLM not only on minority samples but also on interpolated samples to further enrich variability.

Extensive experiments on 10 tabular datasets demonstrate that our method significantly outperforms eight SOTA baselines. The generated synthetic samples are both realistic and diverse. Moreover, we provide theoretical analysis through an entropy-based perspective, proving that our method encourages diversity in the generated samples.

I. Introduction

Recently, applying modern machine learning (ML) models such as deep learning and LLMs to tabular data has gained a significant attention from both research communities and industrial companies [1], [3], [4], [27]. For example, *Prior Labs* – an AI start-up founded in late 2024 focuses on developing LLM-based solutions for tabular data and has raised $9.3 million from investors[1].

However, there are many challenges when dealing with tabular data. One of them is the *imbalance* problem i.e. some classes have much fewer samples than the others. For example, when dealing with a medical dataset, a disease may be very rare and only affect a small proportion of the population, leading to an imbalanced dataset. This can lead to a biased classifier if trained on the imbalanced dataset, where the classifier struggles to properly learn the characteristics of the minority class and it favors the majority labels in its predictions [7], [15], [31].

[1]https://fortune.com/2025/02/05/prior-labs-9-million-euro-preseed-funding-tabular-data-ai

One of the most popular solutions for the imbalance problem is *oversampling*, which generates more minority samples to rebalance the dataset [7], [14], [31], [32]. To evaluate an oversampling method, we often train a ML classifier on the rebalanced dataset and compute performance scores on a test set. A better score indicates a better oversampling method. Figure 1 illustrates the training and evaluation phases.

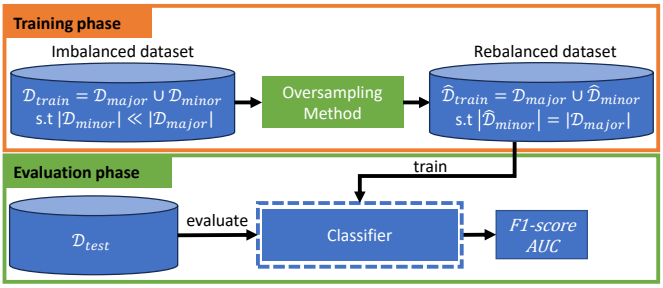


Figure 1: Training and evaluation of an oversampling method. **Training:** the oversampling method learns from the imbalanced dataset $\mathcal{D}_{train}$ to generate a synthetic minority dataset $\hat{\mathcal{D}}_{minor}$ to create the rebalanced dataset $\hat{\mathcal{D}}_{train}$. **Evaluation:** $\hat{\mathcal{D}}_{train}$ is used to train a ML classifier (e.g. XGBoost). The classifier is evaluated on a held-out test set $\mathcal{D}_{test}$ to compute F1-score and AUC. *A better score implies a better oversampling method.*

Most traditional oversampling methods based on SMOTE [7] require the conversion of categorical variables to numeric vectors using ordinal or one-hot encoding. This pre-processing step may cause information loss and artifact introduction [5]. LLM-based methods for oversampling can overcome this problem as they represent tabular data as text instead of numerical. Moreover, they can capture the variable contexts e.g. the relationship between "*Gender*" and "*Job*". To generate a minority sample $\hat{x}$ with M features $\{X_1, ..., X_M\}$, existing LLM-based methods construct a prompt conditioned on the minority label. Namely, they feed a prompt "Y is y_{minor}" (Y is the target variable and y_{minor} is the minority label) as the initial tokens for LLMs to query the next token. They then iteratively concatenate the predicted token to the initial prompt and query the LLMs until all the features are

generated [21], [32]. This process is straightforward as LLM-based methods support *arbitrary conditioning* i.e. the capacity to generate data conditioned on an arbitrary set of variables. However, LLM-based methods often do not generate diverse and generalizable minority samples, which is very important for training classifiers in downstream tasks (see Figure 1).

To address this problem, we propose an LLM-based method with three contributions. First, we observe that existing LLM-based methods construct prompts based on the minority label solely [5], [21], [32]. Although this is simple and intuitive, it generates less diverse synthetic minority samples due to the softmax function used to predict the next token. We propose a simple yet effective trick to fix this issue, which *forms the prompts based on both minority label and features*. Second, the current LLM-based methods permute both the features and the target variable in the fine-tuning data [5], [32]. As decoder-only LLMs use an attention mask to limit the interaction among tokens, there are no attention scores between the minority label and some features if the minority label is shuffled to the middle of the sequence. We address this problem by *permuting only the features while fixing the minority label at the beginning*. Finally, after generating minority samples $\hat{x}$, other LLM-based methods often require a verification process to predict again the labels of synthetic minority samples using either an external classifier [32] or the fine-tuned LLM [21], [31]. However, this step is often unreliable due to the poor classifier/LLM trained/fine-tuned with the imbalanced dataset. We fix it by *fine-tuning the LLM with only minority samples and interpolated samples*.

To summarize, our main contributions are as follows:

1) We propose **ImbLLM**, a novel LLM-based oversampling method that introduces three key improvements—sampling, permutation, and fine-tuning—to generate diverse and robust minority samples.
2) We conduct extensive experiments on 10 tabular datasets against eight SOTA baselines. ImbLLM achieves the best performance in 5 cases and ranks second in 3 cases.
3) Beyond improving imbalanced classification, we demonstrate that the generated samples are superior in both quality and diversity.
4) We provide a theoretical analysis through an entropy-based perspective, formally showing that our generation process promotes diversity.

II. Background

A. Oversampling for imbalanced datasets

Given an *imbalanced* dataset $\mathcal{D}_{train} = \{x_i, y_i\}_{i=1}^N$, each row is a pair of a sample x_i with M features $\{X_1, ..., X_M\}$ and a label y_i (a value of the target variable Y). To simplify the problem, we consider Y as a binary variable i.e. $y_i \in \{0, 1\}$ and $\mathcal{D}_{train}$ contains a set of *majority* samples $\mathcal{D}_{major}$ and a set of *minority* samples $\mathcal{D}_{minor}$, where $| \mathcal{D}_{minor} | \ll | \mathcal{D}_{major} |$. In case the dataset has multiple classes, we define the majority class as the class with the highest number of samples and the minority class as the class with the lowest number of samples.

Our goal is to learn a data synthesizer G from $\mathcal{D}_{train}$, then use G to generate *synthetic* minority samples $\hat{\mathcal{D}}_{minor}$ such that $| \hat{\mathcal{D}}_{minor} | = | \mathcal{D}_{major} |$. Following other works [7], [31], [32], we use $\hat{\mathcal{D}}_{minor}$ to create the *rebalanced* dataset $\hat{\mathcal{D}}_{train} = \mathcal{D}_{major} \cup \hat{\mathcal{D}}_{minor}$. Finally, we evaluate the quality of $\hat{\mathcal{D}}_{minor}$ by measuring the F1 and AUC scores of a ML classifier trained on $\hat{\mathcal{D}}_{train}$ and tested on a held-out dataset $\mathcal{D}_{test}$ (shown in Figure 1). A better score means a better $\hat{\mathcal{D}}_{minor}$.

B. LLM-based methods for oversampling

Existing LLM-based methods for oversampling [5], [21], [31], [32] have three steps: (1) fine-tuning a pre-trained LLM, (2) generating synthetic minority samples, and (3) verifying generated minority samples.

1) Fine-tuning: First, these methods convert each sample x_i and its label y_i into a *sentence*. Although there are several ways to construct a sentence [16], they often transform the ith row $r_i = [X_1 = v_{i,1}, ..., X_M = v_{i,M}, Y = y_i]$ into a corresponding sentence $s_i =$ "X_1 is $v_{i,1}, ..., X_M$ is $v_{i,M}, Y$ is y_i", where $\{X_1, ..., X_M\}$ are feature names, $v_{i,j}$ is the value of the jth feature of the ith row, Y is the target variable and y_i is its value. For example, the row $[Edu = Bachelor, Job = Sales, WH = 35.5, Income < 200K]$ is converted into "*Edu is Bachelor, Job is Sales, WH is 35.5, Income is <200K*". Note that "*WH*" stands for "Working hours per week".

Second, as there is no spacial locality between the columns in the imbalanced dataset, they permute both the features X and the target variable Y. We call this strategy *permute_xy*. Given a sentence $s_i =$ "X_1 is $v_{i,1}, ..., X_M$ is $v_{i,M}, Y$ is y_i", let $a_{i,j} =$ "X_j is $v_{i,j}$" with $j \in \{1, ..., M\}$ and $a_{i,M+1} =$ "Y is y_i", then the sentence can be presented as $s_i =$ "$a_{i,1}, ..., a_{i,M}, a_{i,M+1}$". They apply a *permutation function* P to randomly shuffle the order of X and Y, which results in $s_i =$ "$a_{i,k_1}, ..., a_{i,k_M}, a_{i,k_{M+1}}$", where $[k_1, ..., k_M, k_{M+1}] = P([1, ..., M, M+1])$. For example, the sentence "*Edu is Bachelor, Job is Sales, WH is 35.5, Income is <200K*" is permuted to "*Job is Sales, Edu is Bachelor, Income is <200K, WH is 35.5*".

Finally, they fine-tune a pre-trained LLM following an auto-regressive manner. Given the imbalanced dataset in the text format $\mathcal{S} = \{s_i\}_{i=1}^N$, for each sentence $s_i \in \mathcal{S}$, they tokenize it into a sequence of tokens $(c_1, ..., c_l) = \text{tokenize}(s_i)$. They factorize the probability of a sentence s_i into a product of output probabilities conditioned on previously observed tokens:

$$p(s_i) = p(c_1, ..., c_l) = \prod_{k=1}^{l} p(c_k \mid c_1, ..., c_{k-1}) \quad (1)$$

They train the model using maximum likelihood estimation (MLE), which maximizes the probability $\prod_{s_i \in \mathcal{S}} p(s_i)$ of the entire training dataset $\mathcal{S}$.

2) Sampling $\hat{x}$: First, they use the minority label "Y is y_{minor}" as a *condition* to query the fine-tuned LLM Q e.g. "*Income is* $\geq$ 200*K*". This condition is converted into a sequence of tokens $(c_1, ..., c_{k-1})$. Second, they sample the next

token c_k from a conditional probability distribution defined by a *softmax* function:

$$p(c_k \mid c_1, ..., c_k) = \frac{e^{(z_{c_k}/T)}}{\sum_{c' \in \mathcal{C}} e^{(z_{c'}/T)}}, \quad (2)$$

where $e()$ is an exponential function, $z = Q(c_1, ..., c_{k-1})$ are the logits over all possible tokens, $T > 0$ is a temperature, and $\mathcal{C}$ is the vocabulary of tokens.

This step is repeated until all the features are sampled to generate a synthetic minority sample $\hat{x}$. We call this strategy *condition_y*.

3) Verifying $\hat{x}$: For a standard LLM-based method like Great [5], the process to generate $\hat{x}$ is stopped at step (2). Some methods employ the third step to verify generated minority samples. Given a synthetic minority sample $\hat{x}_i = [X_1 = v_{i,1}, ..., X_M = v_{i,M}]$, TapTap [32] uses a classifier trained with $\mathcal{D}_{train}$ to predict the label $\hat{y}_i$ for $\hat{x}_i$ again. Meanwhile, Pred-LLM [21] and LITO [31] construct a prompt based on $\hat{x}_i$ as "X_1 is $v_{i,1}, ..., X_M$ is $v_{i,M}$" and use it as a condition to query the fine-tuned LLM Q for the label $\hat{y}_i$. If $\hat{y}_i$ is predicted as a majority label, they discard $\hat{x}_i$.

III. Framework

A. The proposed method

We propose an LLM-based method (called **ImbLLM**) to generate minority samples that mimic the real minority samples. ImbLLM improves the diversity of synthetic minority samples through the three following proposals.

1) Sampling $\hat{x}$ conditioned on both minority label and feature: As described in the section Background II-B, generating synthetic minority samples $\hat{x}$ is straightforward for LLM-based methods. By forming the minority label as initial tokens to query the fine-tuned LLM Q, we can generate more minority samples, which oversample the minority class and rebalance the dataset. However, there are two problems with this approach we need to solve.

First problem: *The synthetic minority samples may not be diverse*. Assume that we want to synthesize 1,000 minority samples, then we need to create 1,000 prompts where each of them is the same sentence "Y is y_{minor}", where Y is the target variable and y_{minor} is the minority label e.g. "*Income is* $\geq$ 200*K*". We use Equation (2) to generate the next token. We call this strategy *condition_y*. As it is a softmax function, it may always return the token with the highest probability. In other words, given 1,000 identical prompts, we may receive very similar 1,000 synthetic minority samples.

To solve this problem, we propose a simple method that constructs diverse prompts using both minority label and features. For each prompt "Y is y_{minor}", we first uniformly sample a feature X_i from the list of features $\{X_1, ..., X_M\}$, then sample a value from the distribution of X_i (i.e. $v_i \sim p(X_i)$), finally concatenate "X_i is v_i" with "Y is y_{minor}". Our prompt has a new form of "Y is y_{minor}, X_i is v_i" e.g. "*Income is* $\geq$ 200*K, Edu is Bachelor*". Our prompts are much more diverse as they combine the minority label with different features and values. We call our strategy *condition_yx*.

Second problem: *Using the minority label as initial tokens may not generate diverse samples due to the permutation strategy permute_xy*. Recall that *permute_xy* is used in existing LLM-based methods to shuffle both the features X and the target variable Y (see the section Background II-B). As these methods use decoder-only LLMs, they use *attention matrices* to determine the influence among tokens. Formally, given a sequence of tokens $(c_1, ..., c_l)$, let $\mathbf{E} \in \mathbb{R}^{l \times d}$ be the embedding matrix of the tokens at any given transformer layer. The self-attention mechanism computes the query and key matrices $\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{l \times d_k}$ using a learned linear transformation:

$$\mathbf{Q} = \mathbf{E}\mathbf{W}_Q, \mathbf{K} = \mathbf{E}\mathbf{W}_K, \quad (3)$$

where $\mathbf{W}_Q, \mathbf{W}_K$ are weight matrices learned by the transformer model, and d, d_k are dimensions of embedding and query/key.

The attention matrix is computed as:

$$\mathcal{A} = \text{softmax}(\frac{\mathcal{M}_{\{i \leq j\}}\mathbf{Q}\mathbf{K}^{\mathrm{T}}}{\sqrt{d_k}}) \quad (4)$$

In Equation (4), decoder-only LLMs use an attention mask $\mathcal{M}_{\{i \leq j\}}$ to limit token interactions, which only allows each token is influenced by its preceding tokens. Namely, given a current token i, $\mathcal{M}_{\{i \leq j\}} = \mathbb{I}_{i \leq j}$ will set the attention score to 0 if the token j is on the left of i.

As existing LLM-based methods permute both X and Y, the minority label y_{minor} can be shuffled to the middle/end of the sentence. Due to the attention mask, there is no attention score between y_{minor} and some features before it. When we use y_{minor} as a condition to sample other features, LLMs have only few choices to generate the next token, leading to less diverse synthetic samples.

2) Permuting X but fixing Y: To address the above problem, we propose the permutation strategy *fix_y*, which only permutes X while fixing Y at the beginning of the sentence. Formally, given a sentence $s_i =$ "$a_{i,1}, ..., a_{i,M}, a_{i,M+1}$", we apply the permutation function P to M features and move Y to the beginning, which results in a permuted sentence $s_i =$ "$a_{i,M+1}, a_{i,k_1}, ..., a_{i,k_M}$", where $[k_1, ..., k_M] = P([1, ..., M])$. For example, the sentence "*Edu is Bachelor, Job is Doctor, WH is 40.2, Income is* $\geq$ 200*K*" is permuted to "*Income is* $\geq$ 200*K, Edu is Bachelor, WH is 40.2, Job is Doctor*". Note that "*Income is* $\geq$ 200*K*" is at the beginning of the permuted sentence.

We illustrate the attention matrices derived by *permute_xy* (other methods) and *fix_y* (our method) in Figure 2. Figure 2(a) shows that using *permute_xy*, the minority label "$\geq$ 200*K*" has only attention score with the feature "*WH*", and the LLM is likely to predict "*WH is 40.2*" after the condition "*Income is* $\geq$ 200*K*". In contrast, Figure 2(b) shows that our *fix_y* helps the minority label "$\geq$ 200*K*" to retrieve attention scores with all features, and the LLM can predict any feature after the condition, resulting in more diverse synthetic samples.

3) Fine-tunning LLMs with minority samples and their interpolation: As existing LLM-based methods use both majority and minority samples to fine-tune the LLM, the synthetic

	Job	Doctor	...	Income	≥200K	WH	40.2
Job	0	0.5	...	0.15	0.25	0.45	0.15
Doctor	0	0	...	0.45	0.35	0.25	0.05
...	...	...	...	...	...	...	...
Income	0	0	...	0	0.8	0.22	0.15
≥200K	0	0	...	0	0	0.44	0.33
WH	0	0	...	0	0	0	0.72
40.2	0	0	...	0	0	0	0

(a) *permute_xy*

	Income	≥200K	...	WH	40.2	Job	Doctor
Income	0	0.6	...	0.3	0.2	0.4	0.1
≥200K	0	0	...	0.45	0.3	0.5	0.15
...	...	...	...	...	...	...	...
WH	0	0	...	0	0.7	0.2	0.05
40.2	0	0	...	0	0	0.4	0.35
Job	0	0	...	0	0	0	0.55
Doctor	0	0	...	0	0	0	0

(b) *fix_y*

Figure 2: Attention matrix. Existing LLM-based methods use *permute_xy* to shuffle both X and Y, leading to no attention scores between y_{minor} and some features (a). Our method uses *fix_y* to permute only X while fixing Y at the beginning, resulting in attention scores between y_{minor} and all features (b).

minority samples $\hat{x}$ may be biased to the majority label although they are already conditioned on the minority label [31]. They address this problem by using either an external classifier [32] or the fine-tuned LLM itself [21], [31] to verify $\hat{x}$. But this strategy raises another problem. Since the classifier/LLM is trained/fine-tuned with the imbalanced dataset $\mathcal{D}_{train}$, we expect that their performance is not optimal, leading to an unreliable verification step. As we show in the experiments, this verification does not work on nearly half of the benchmark datasets.

We propose a simple trick to make sure that the generated samples are indeed minority samples without involving any verification. Instead of using both majority and minority samples, we use only minority samples to fine-tune the LLM. This helps our LLM to focus on generating minority samples while it is not influenced by (or biased to) majority samples. However, it has a weakness. The number of possible values of a continuous variable is often much larger than that of a categorical variable. For example, the categorical variable "*Edu*" has only few values {"School", "Bachelor", "Master", "PhD"} whereas the continuous variable "*WH*" can have any continuous value in $[0, 168]$. Thus, if we use $\mathcal{D}_{minor}$ (a small dataset), although we have only few samples, their categorical values are still good enough to cover the categorical domains. However, their continuous values are too limited to capture the characteristics of the continuous domains.

Interpolation. We address this issue with an interpolation step. First, we randomly sample a minority sample $x_i \in \mathcal{D}_{minor}$. Let x_i^{con} and x_i^{cat} be continuous and categorical values of x_i. Then, we interpolate x_i using:

$$x_i' = x_i^{con} + \epsilon(x_j^{con} - x_i^{con}), \tag{5}$$

where $x_j \in \mathcal{D}_{minor}$ is another random minority sample and $\epsilon \in [0, 1]$ is a random number. Here, we only interpolate continuous variables of x_i. After the interpolation process, we obtain a set of *interpolated samples* $\mathcal{D}_{inter} = \{x_i', y_i'\}_{i=1}^{|\mathcal{D}_{major}|}$. Note that a minority sample $x_i \in \mathcal{D}_{minor}$ has M features but an interpolated sample $x_i' \in \mathcal{D}_{inter}$ has only M_{con} features where M_{con} and M_{cat} are the numbers of continuous and categorical variables and $M_{con} + M_{cat} = M$. We generate the set of interpolated samples such that $| \mathcal{D}_{inter} | = | \mathcal{D}_{major} |$. Finally, we fine-tune our LLM with $\mathcal{D}_{train} = \mathcal{D}_{minor} \cup \mathcal{D}_{inter}$.

The workflow of our method ImbLLM is illustrated in Figure 3.

IV. Experiments

We conduct extensive experiments to show that our method is better than other methods under different qualitative and quantitative metrics.

A. Experiment settings

1) Datasets: We evaluate our method on 10 real-world tabular datasets. They are commonly used in classification tasks [5], [30], [32]. The details of these datasets are provided in Table I.

Table I: Dataset characteristics. N, M, *#Con*, *#Cat*, and K indicate the numbers of samples, features, continuous variables, categorical variables, and classes.

Dataset	N	M	*#Con*	*#Cat*	K	Source
adult	30,162	11	5	6	2	UCI
german	1,000	20	6	14	2	UCI
fuel	639	8	3	5	2	Kaggle
insurance	1,338	6	3	3	2	Kaggle
gem_price	26,270	9	6	3	2	Kaggle
housing	20,433	9	8	1	2	Kaggle
bank	4,521	14	5	9	2	[22]
car	1,728	6	2	6	4	UCI
credit_card	10,127	19	14	5	2	UCI
sick	3,560	27	6	21	2	OpenML

2) Evaluation metric: To evaluate the performance of oversampling methods, we use the synthetic minority samples to rebalance the imbalanced dataset similar to other works [5], [30], [32]. For each dataset, we randomly split it into 80% for the training set $\mathcal{D}_{train}$ and 20% for the test set $\mathcal{D}_{test}$. To construct an imbalanced dataset, we reduce the number of minority samples like other works [31], [32], where we only use 20% minority samples. Formally, the *imbalanced* training set $\mathcal{D}_{train} = \mathcal{D}_{major} \cup \mathcal{D}_{minor}$, where $| \mathcal{D}_{minor} | \ll | \mathcal{D}_{major} |$ and $| \mathcal{D}_{minor} | = q \times | \mathcal{D}_{minor}^* |$ with $q \in (0, 1]$ being the *imbalance-ratio* and $\mathcal{D}_{minor}^*$ being the *original* set of minority samples (before reduced). We set $q = 0.2$ for all experiments.

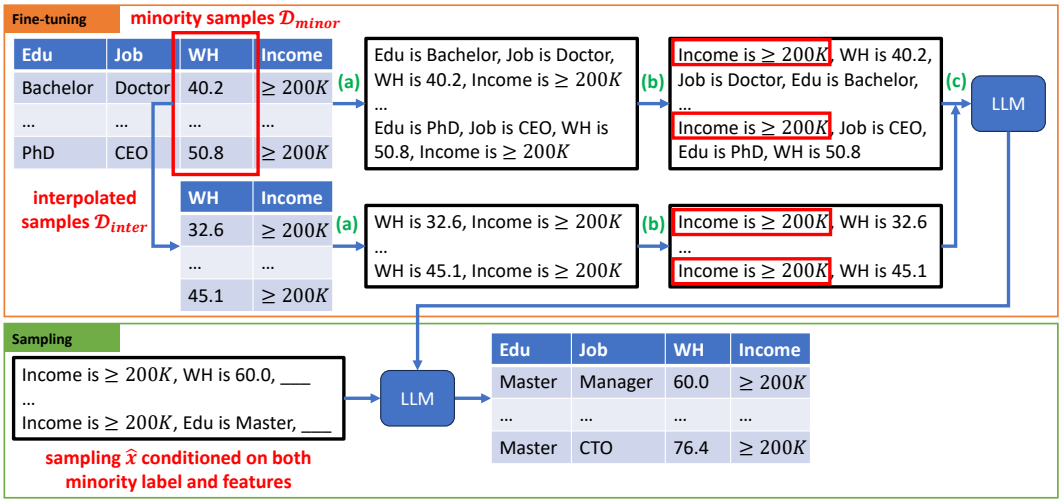


Figure 3: Overview of our ImbLLM. **First step:** it converts each row to a sentence (a), permutes X and fixes Y at the beginning (b), and fine-tunes an LLM with minority samples $\mathcal{D}_{minor}$ and interpolated samples $\mathcal{D}_{inter}$ (c). **Second step:** it constructs prompts based on both minority label and features to query the fine-tuned LLM to generate minority samples $\hat{x}$.

We then train oversampling methods with the *imbalanced* dataset $\mathcal{D}_{train}$ to generate the set of synthetic minority samples $\hat{\mathcal{D}}_{minor}$ such that $\mid \hat{\mathcal{D}}_{minor} \mid = \mid \mathcal{D}_{major} \mid$. Finally, we train the ML classifier XGBoost on the *rebalanced* dataset $\hat{\mathcal{D}}_{train} = \mathcal{D}_{major} \cup \hat{\mathcal{D}}_{minor}$ and compute its F1-score and AUC on $\mathcal{D}_{test}$. We repeat each method *three times with random seeds* and report the average score along with its standard deviation.

We also compute other metrics between the *original* minority samples $\mathcal{D}^*_{minor}$ and the *synthetic* minority samples $\hat{\mathcal{D}}_{minor}$ to measure the *quality* and *diversity* of $\hat{\mathcal{D}}_{minor}$. Note that $\mathcal{D}^*_{minor}$ is ***unseen*** for all oversampling methods. It is just used for the evaluation purpose.

(1) *Close probability* [24]: We compute the probability of *close values* between $\mathcal{D}^*_{minor}$ and $\hat{\mathcal{D}}_{minor}$. For each real minority sample $x \in \mathcal{D}^*_{minor}$, we compute its distance to the closest synthetic minority sample $\hat{x} \in \hat{\mathcal{D}}_{minor}$, and normalize the distance $d(x, \hat{x})$ to $[0, 1]$. Given a threshold $\alpha = 0.2$, we define x has a close value if $d(x, \hat{x}) \leq \alpha$. We compute the close probability as $Close = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}_{d(x_i, \hat{x}_i) \leq \alpha}$. This score measures the *quality* of $\hat{\mathcal{D}}_{minor}$, where 0 means synthetic samples are very dissimilar to real samples whereas 1 means all synthetic samples are similar to some real samples. *A higher score is better.*

(2) *Coverage* [20]: We compute the coverage score between $\mathcal{D}^*_{minor}$ and $\hat{\mathcal{D}}_{minor}$, which measures the fraction of real samples whose neighborhoods contain at least one synthetic sample. $Cov = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}_{\exists j \text{ s.t. } \hat{x}_j \in B(x_i, d_k(x_i))}$, where $B(x_i, d_k(x_i))$ is the sphere around x_i and the radius $d_k(x_i)$ is the Euclidean distance from x_i to its k-th nearest neighbor (we set $k = 2$). It measures the *diversity* of $\hat{\mathcal{D}}_{minor}$ as it shows how the synthetic minority samples resemble the variability of the real minority samples. *A higher score is better.*

3) Baselines: We compare with eight oversampling baselines, including traditional methods (*AdaSyn* [14], *SMOTE* [7], and *SMOTE-NC* [7]), generative-based methods (*CTGAN* [30] and *TVAE* [30]), and LLM-based methods (*Great* [5], *TapTap* [32], and *Pred-LLM* [21]). For reproducibility, we use the published source codes of AdaSyn, SMOTE, and SMOTE-NC[2], TVAE and CTGAN[3], Great[4], TapTap[5], and Pred-LLM[6]. We do not compare with the LLM-based method LITO [31] as its source code is not available.

The *Original* method means the classifier trained with the *original* majority and minority sets $\mathcal{D}_{major}$ and $\mathcal{D}^*_{minor}$. As $\mathcal{D}^*_{minor}$ is large and just slightly imbalanced from $\mathcal{D}_{major}$, we can expect the Original method achieves a good result, which can be served as the reference. The *Imbalance* method means the classifier trained directly with the *imbalanced dataset* $\mathcal{D}_{train}$. Other methods mean the classifiers trained with the *rebalanced dataset* $\hat{\mathcal{D}}_{train}$ after performing the oversampling step. We use XGBoost for the classifier since it is one of the most popular predictive models for tabular data [27]. Following [5], [21], [32], we use the distilled version of ChatGPT-2 for our LLM, train it with *batch_size=32*, *#epochs=50*, and set $T = 0.7$ for Equation (2).

B. Results and discussions

1) Imbalanced classification task: Table II reports F1-score of each oversampling method on 10 benchmark datasets. *We recall that the procedure to compute the score is described in Figure 1.* Our method ImbLLM performs better than other methods. Among 10 datasets, ImbLLM is the best performing on five datasets and the second-best on another three datasets. The average improvement of ImbLLM over CTGAN (the

[2]https://imbalanced-learn.org/stable/
[3]https://github.com/sdv-dev/CTGAN
[4]https://github.com/kathrinse/be_great
[5]https://github.com/ZhangTP1996/TapTap
[6]https://github.com/nphdang/Pred-LLM

runner-up method) is around 3% and Imbalance (the standard classifier without oversampling) is around 8%.

As we discussed earlier, traditional methods AdaSyn and SMOTE perform poorly as they require a conversion of categorical variables to numerical vectors, leading to information loss. This is a serious problem as many experimental datasets have a large number of categorical variables. SMOTE-NC (it is an improved version of SMOTE) addresses mixed types much better and becomes the third-best method, following CTGAN. The LLM-based methods are better than SMOTE and they are comparable with TVAE. The results also show that TapTap and Pred-LLM with the verification step via either an external classifier or the fine-tuned LLM significantly outperform the standard LLM method Great. However, thanks to the diversity of our synthetic minority samples, ImbLLM greatly improves over TapTap and Pred-LLM by 9% and 7%.

Most oversampling methods improve over Imbalance (the method trains the classifier on the imbalanced dataset without rebalancing it). However, their performance is still far away from the Original method. Only our ImbLLM can reach to the performance level of Original (0.8002 vs. 0.8238) although ImbLLM only uses 20% minority samples of Original.

The AUC scores of top-3 methods are reported in Figure 4, which also shows ImbLLM is the best method.

2) Quality and diversity evaluation: As described in Section IV-A2, we compute the Close probability and Coverage scores to measure the *quality and diversity of synthetic minority samples*. Figure 4 presents the average scores of top-3 methods on all datasets. Our method ImbLLM significantly outperforms other methods for all metrics. The quality metric Close probability shows that our synthetic minority samples are closer to the original minority samples than other synthetic samples (0.8011 vs. 0.7761 of CTGAN). In terms of diversity, the Coverage score shows that our synthetic minority samples are more diverse than others (0.8933 vs. 0.8482 of CTGAN).

High scores in both quality and diversity metrics show our synthetic minority samples can capture the *real manifold of minority samples*, which explains their benefits in imbalanced classification tasks (see Table II and "AUC" in Figure 4).

C. Ablation studies

We analyze our method under different configurations.

1) Effect investigation of various modifications: We have three novel contributions in *sampling*, *permutation*, and *fine-tuning*. Table III reports the F1-score of each modification compared to that of the LLM baseline Great. Recall that Great generates minority samples conditioned on the minority label (denoted by *condition_y*), permutes both the features X and the target variable Y (denoted by *permute_xy*), and fine-tunes the LLM with both majority and minority samples (denoted by *major+minor*).

While Great only achieves 0.6833, our modifications show improvements. By fixing the target variable Y at the beginning of the sentence (denoted by *fix_y*) or fine-tuning the LLM with only minority samples and their interpolation (denoted by *minor+interpolate*), we achieve F1-score up to $\sim$ 0.74 (i.e. 6% improvement). Combining these two proposals with our diversity sampling strategy (denoted by *condition_yx*), we achieve the best result at 0.8002 (the last row). This study suggests that each modification is useful, which significantly improves Great.

2) Effect investigation of interpolation: As discussed earlier, we fine-tune the LLM using only minority samples to avoid biased generated minority samples. This also helps us to skip the verification step via either an external classifier (used in TapTap) or the LLM itself (used in Pred-LLM). However, as the minority set is small, the synthetic minority samples may not be generalizable to cover *unseen* minority samples, leading to poor performance in imbalanced classification tasks. We address this problem with interpolated samples (see Section III-A3). We show the benefit of the interpolation by comparing ImbLLM with its variation *ImbLLM-inter* (i.e. we keep all components in ImbLLM except the interpolation step). The results are summarized in Table IV, where ImbLLM is better than ImbLLM-inter on most datasets in terms of both classification performance and quality. Interestingly, ImbLLM-inter is still better than CTGAN (the best baseline) on all scores, which proves our contributions are very effective and robust.

We also investigate the impact of the number of interpolated samples on our method's performance. Recall that in Section III-A3, we generate the set of interpolated samples $\mathcal{D}_{inter} = \{x'_i, y'_i\}_{i=1}^{|\mathcal{D}_{major}|}$, where the number of interpolated samples $| \mathcal{D}_{inter} | = | \mathcal{D}_{major} |$. Here, we control the size of $\mathcal{D}_{inter}$ by employing an *interpolation-ratio* $r \in [0, 1]$ such that $| \mathcal{D}_{inter} | = r \times | \mathcal{D}_{major} |$.

Figure 5 shows F1-scores of our method ImbLLM with different values for r, compared with the best baseline CTGAN. Note that at $r = 0$, we have the performance of ImbLLM-inter (the version of ImbLLM without interpolation) and at $r = 1$, we have the full performance of our method ImbLLM. We can see that our performance increases when we set larger values for r, and our method is always better than CTGAN with all values for r.

The results prove that our interpolation step is very effective to generate diverse and generalizable synthetic minority samples, resulting in better outcomes for imbalanced classification tasks.

3) Effect investigation of imbalance-ratio: Given the original set of minority samples $\mathcal{D}^*_{minor}$, we use $| \mathcal{D}_{minor} | = q \times | \mathcal{D}^*_{minor} |$ as the number of minority samples in the training set of oversampling methods. $q \in (0, 1]$ is the imbalance-ratio, we set $q = 0.2$ in our experiments. In this study, we adjust q to see whether it has impacts on our performance.

Figure 6 shows our method ImbLLM improves when the number of minority samples increases. It improves significantly when we enlarge q from 0.1 to 0.5 (the improvement increases from 0.7565 up to 0.8242). Other methods also benefit from more training minority samples, e.g. Imbalance improves from 0.6693 up to 0.7902. When the imbalance-ratio is severe (i.e. $q \in [0.1, 0.3]$), there is a big gap between our performance and those of the baselines. However, the gap

Table II: F1-score ± (std) of each oversampling method on 10 datasets. **Bold** and underline indicate the best and second-best methods.

F1-score	Original	Imbalance	AdaSyn	SMOTE	SMOTE-NC	CTGAN	TVAE	Great	TapTap	Pred-LLM	ImbLLM
adult	0.6761 (0.056)	0.4685 (0.030)	0.3987 (0.000)	0.3987 (0.000)	0.5586 (0.056)	0.5223 (0.013)	0.4985 (0.041)	**0.6592** (0.040)	0.4705 (0.024)	0.5071 (0.059)	0.6543 (0.044)
german	0.5299 (0.080)	0.2032 (0.076)	**0.4601** (0.002)	0.4595 (0.003)	0.2706 (0.127)	0.4064 (0.039)	0.2760 (0.124)	0.2059 (0.148)	0.2069 (0.060)	0.2363 (0.034)	0.4419 (0.053)
fuel	0.9645 (0.026)	0.9664 (0.018)	0.9458 (0.038)	0.9406 (0.041)	0.9662 (0.009)	0.9658 (0.009)	0.9642 (0.019)	0.9272 (0.017)	0.9751 (0.011)	0.9662 (0.010)	**0.9820** (0.017)
insurance	0.9153 (0.005)	0.9220 (0.006)	0.3933 (0.054)	0.4263 (0.027)	**0.9253** (0.005)	0.8858 (0.034)	0.9193 (0.009)	0.7690 (0.016)	0.9209 (0.012)	0.9182 (0.010)	0.9250 (0.019)
gem_price	0.9799 (0.002)	0.9707 (0.002)	0.2818 (0.158)	0.5931 (0.147)	0.9739 (0.001)	0.9705 (0.002)	0.9711 (0.001)	0.9698 (0.003)	0.9709 (0.002)	0.9733 (0.002)	**0.9755** (0.001)
housing	0.9111 (0.002)	0.8738 (0.006)	0.4421 (0.005)	0.4450 (0.005)	0.8801 (0.011)	0.8690 (0.011)	0.8707 (0.016)	0.8721 (0.009)	0.8756 (0.005)	**0.8826** (0.009)	0.8750 (0.011)
bank	0.3657 (0.035)	0.1118 (0.022)	0.2063 (0.000)	0.2064 (0.000)	0.2322 (0.077)	0.3162 (0.071)	0.2134 (0.047)	0.2183 (0.025)	0.0831 (0.029)	0.1352 (0.102)	**0.3441** (0.028)
car	0.9923 (0.004)	0.9757 (0.011)	0.7695 (0.006)	0.7689 (0.007)	0.9684 (0.010)	0.9787 (0.005)	0.9734 (0.007)	0.5732 (0.034)	0.9716 (0.014)	**0.9831** (0.005)	0.9820 (0.004)
credit_card	0.9825 (0.002)	0.9671 (0.002)	0.0436 (0.015)	0.0701 (0.024)	0.9725 (0.001)	0.9698 (0.001)	0.9671 (0.003)	0.9634 (0.002)	0.9634 (0.001)	0.9692 (0.002)	**0.9758** (0.002)
sick	0.9208 (0.032)	0.7790 (0.012)	0.1826 (0.053)	0.2057 (0.061)	0.8270 (0.009)	0.8394 (0.040)	0.8376 (0.037)	0.6752 (0.044)	0.7386 (0.026)	0.7740 (0.030)	**0.8459** (0.004)
Average	0.8238	0.7238	0.4124	0.4514	0.7575	0.7724	0.7491	0.6833	0.7177	0.7345	**0.8002**

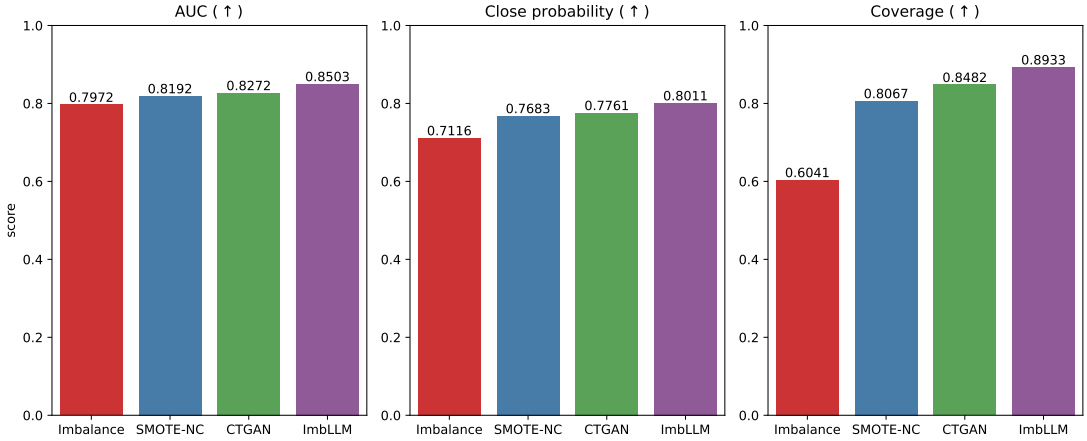


Figure 4: The average quality and diversity scores. ↑ means "*higher is better*".

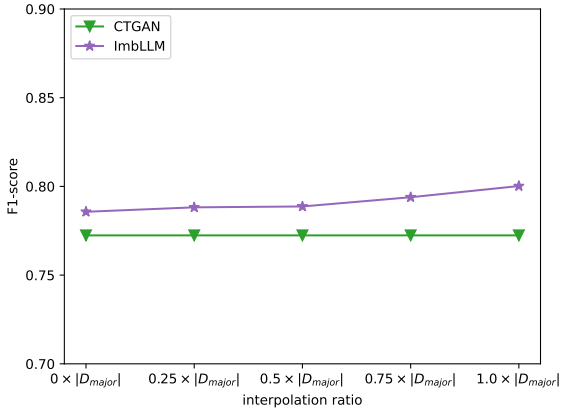


Figure 5: F1-score vs. interpolation-ratio over 10 datasets.

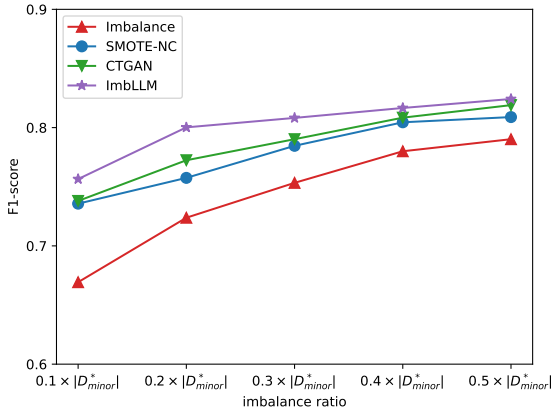


Figure 6: F1-score vs. imbalance-ratio over 10 datasets.

Table III: Effectiveness of different modifications in ImbLLM. The F1-score is averaged over 10 datasets. "✓" indicates ImbLLM uses the same setting as Great.

	Sampling $\hat{x}$	Permutation	Fine-tuning	**F1-score**
Great	*condition_y*	*permute_xy*	*major+minor*	0.6833
ImbLLM	*condition_yx*	✓	✓	0.6690
	✓	*fix_y*	✓	0.7408
	✓	✓	*minor+interpolate*	0.7436
	condition_yx	*fix_y*	✓	0.7181
	condition_yx	✓	*minor+interpolate*	0.7998
	✓	*fix_y*	*minor+interpolate*	0.7421
	condition_yx	***fix_y***	***minor+interpolate***	**0.8002**

is smaller when more minority samples are presented in the training set, as expected.

4) Weakness of TapTap and Pred-LLM: TapTap and Pred-LLM are two recent LLM methods that can effectively generate synthetic minority samples. They involve an extra step to verify and reject ill-generated minority samples. Namely, they use a classifier/LLM trained/fine-tuned on the imbalanced dataset $\mathcal{D}_{train}$ to predict again the labels of synthetic minority samples. However, this strategy can work well only in the case the imbalanced dataset can represent the original dataset. In other words, if the classifier trained on the imbalanced dataset (i.e. the Imbalance method) has a poor performance, it may wrongly reject synthetic minority samples. As shown in Table V, the Original method trained with the original dataset has high F1-score. However, the Imbalance method has performance reduced a lot on these four datasets, leading to an ineffective verification step. As a result, TapTap and Pred-LLM perform poorly. In contrast, as ImbLLM does not rely on the verification step, it can reach to much better F1-scores.

D. Generalization and diversity visualization

We plot the *distance to closest records* (DCR) histogram [5] to show *our synthetic minority samples are similar to* ***unseen*** *minority samples in* $\mathcal{D}_{test}$. The DCR metric computes the distance from a real minority sample to its closest neighbor in the set of synthetic minority samples. Given a real minority sample $x \in \mathcal{D}_{test}$, $\text{DCR}(x) = \min\{d(x, \hat{x}_i) \mid \hat{x}_i \in \hat{\mathcal{D}}_{minor}\}$, where $d(\cdot)$ is an Euclidean distance.

Figure 7 visualizes the DCR distribution of each method. Besides the top-3 methods, we also include the best two LLM-based methods TapTap and Pred-LLM in the comparison list. Only our method ImbLLM can generate synthetic minority samples in close proximity to the *unseen* minority samples. Other methods show differences. Namely, most of our synthetic samples have the distances to the unseen samples around 0 while most of synthetic samples of other methods have the distances to the unseen samples around 1. This result proves that our synthetic samples are diverse and generalizable to cover many unseen samples.

V. Related Works

A. Machine learning for tabular data

Tabular data is one of the most popular data formats. In the "AI Report" of Kaggle in 2023, it was estimated that between 50% and 90% of practicing data scientists used tabular data as their primary data type in their experiments[7].

ML and deep learning methods have been applied to tabular data in many tasks. Some examples include: (1) *table question answering* answers a given question over tables [17], (2) *table fact verification* checks if an assumption is valid for a given table [9], (3) *table to text* uses natural language to describe a given table [2], and (4) *table structure understanding* identifies the table characteristics e.g. column types, variable relations... [28], [29].

Among these tasks, *table generation* and *table prediction* (or *table classification*) are the most well-known tasks used in tabular data [1], [4], [5], [12], [16], [21].

B. Generative models for tabular data

As generative models gained significant successes in image generation, they have been adapted for *table generation* (or *tabular data generation*) in different ways. Given a table, table generation aims to learn a synthetic table that approximates the real table.

Most tabular generation methods are based on Generative Adversarial Network (GAN) [11] and Variational Autoencoder (VAE) [19] models, including TableGAN [23], CTGAN [30], OCTGAN [18], and TVAE [30]. Among GAN-based methods, CTGAN [30] is the most favored method because it has three contributions to improve the data generation process. First, it leverages different activation functions to generate continuous and categorical data separately. Second, it normalizes a continuous value based on its mode-specific instead of the mean value or the min-max values of the corresponding column. Finally, it uses a conditional generator to generate equal data for different categories in categorical columns. These proposed steps greatly improve the quality of the synthetic table.

As GAN-based methods require *heavy pre-processing* steps, LLM-based methods were introduced to address this problem [5], [21], [32]. They show three advantages over GAN-based methods: (1) avoiding information loss, (2) allowing context-aware, and (3) supporting arbitrary conditioning. These benefits help them to generate realistic tabular data used in various applications such as data augmentation [26], imbalanced classification [31], and few-shot classification [16].

C. Imbalanced classification in tabular data

Table prediction (or *table classification*) is the most popular task in tabular data. It predicts a label for the target variable (e.g. "*Income*") using a set of features (e.g. "*Age*" and "*Education*"). While deep learning methods are often dominant in other tasks, traditional ML methods like XGBoost [8] still outperform deep learning counterparts in table prediction.

Imbalanced classification is a related task to table classification, where the classes in the training table are not equally represented (i.e. one class has a much larger samples than the others). There are two main approaches to address the class-imbalance. Model-centric approaches focus on modifying the

[7]https://www.kaggle.com/AI-Report-2023

Table IV: ImbLLM vs. ImbLLM-inter (i.e. ImbLLM without interpolation).

	F1-score		AUC		Close probability	
	ImbLLM-inter	ImbLLM	ImbLLM-inter	ImbLLM	ImbLLM-inter	ImbLLM
adult	**0.6792**	0.6543	**0.7904**	0.7662	**0.9989**	0.9982
german	0.3917	**0.4419**	0.6159	**0.6357**	0.9472	**0.9528**
fuel	0.9730	**0.9820**	0.9597	**0.9716**	0.3704	**0.4237**
insurance	0.9118	**0.9250**	0.9025	**0.9135**	**0.5569**	0.4990
gem_price	0.9740	**0.9755**	**0.9724**	0.9723	**0.9941**	0.9925
house	**0.8813**	0.8750	**0.8424**	0.8388	0.9972	**0.9977**
bank	0.2544	**0.3441**	0.5757	**0.6174**	**0.9928**	0.9816
car	0.9768	**0.9820**	0.9626	**0.9683**	0.3526	**0.3782**
credit_card	0.9724	**0.9758**	0.8732	**0.8983**	0.8902	**0.9188**
sick	0.8426	**0.8459**	0.8637	**0.9209**	0.7704	**0.8685**
Average	0.7857	**0.8002**	0.8359	**0.8503**	0.7871	**0.8011**

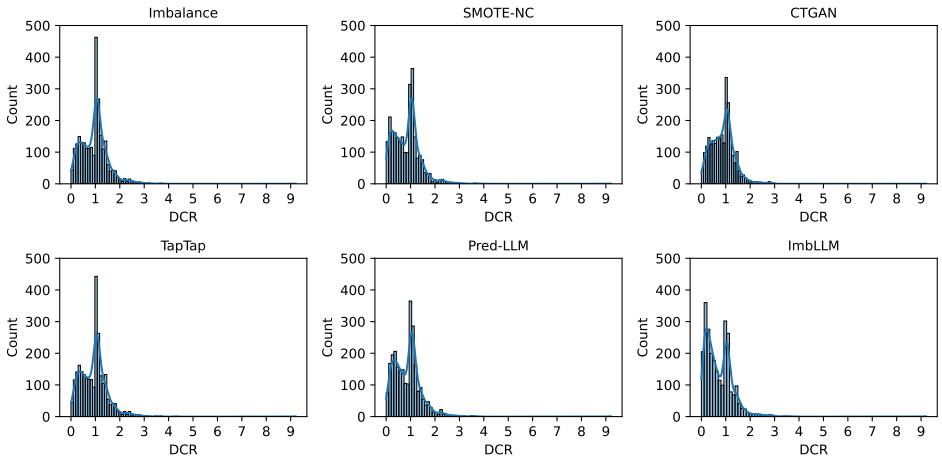


Figure 7: DCR distributions for the dataset *gem_price*. While other methods show that their synthetic minority samples are far away from the *unseen* minority samples (their mode is around 1), our ImbLLM shows that our synthetic minority samples are close to the *unseen* minority samples (our mode is around 0).

Table V: TapTap and Pred-LLM weakness on four datasets. Recall that they use a classifier/LLM trained/fine-tuned on the imbalanced dataset to verify synthetic minority samples. However, when the classifier does not perform well (indicated by the low F1-scores of Imbalance), the verification step is ineffective, leading to poor performance.

F1-score	Original	Imbalance	TapTap	Pred-LLM	ImbLLM
adult	0.6761	0.4685	0.4705	0.5071	**0.6543**
german	0.5299	0.2032	0.2069	0.2363	**0.4419**
bank	0.3657	0.1118	0.0831	0.1352	**0.3441**
sick	0.9208	0.7790	0.7386	0.7740	**0.8459**

objective functions in the ML classifiers [6] or re-weighting the minority class [10]. A data-centric approach is oversampling, which generates more synthetic minority samples.

Oversampling methods are well-studied solutions for imbalanced classification. Their principle is simple but effective. Most existing methods rely on SMOTE (Synthetic Minority Oversampling Technology) [7], where more minority samples are generated by linearly combining two real minority samples to rebalance the dataset. Several variants have been developed to address SMOTE weaknesses such as outlier and noisy [13], [25]. Other approaches are generative models such as CTGAN and TVAE [30], which learn the distribution of real minority samples via a generator or encoder network.

Recently, LLMs have been adapted for oversampling methods [5], [21], [31], [32]. These methods often involve two main steps. First, they fine-tune a pre-trained LLM with the imbalanced dataset. Then, they construct prompts conditioned on the minority label to query the fine-tuned LLM to generate minority samples. However, most of them focus on how to re-verify the synthetic minority data but do not emphasize the *data diversity* that is an important factor in imbalanced classification tasks.

VI. Conclusion

LLM-based methods have emerged as potential solutions for oversampling minority samples. However, they fail to produce diverse synthetic data. To address this problem, we propose an LLM-based oversampling method (named **ImbLLM**) with three important proposals in *sampling*, *permutation*, and *fine-tuning*. Our method shows significant improvements over eight SOTA baselines on 10 tabular datasets in imbalanced classification tasks. It also shows our synthetic minority samples have high quality and diversity based on qualitative and quantitative metrics and visualization.

REFERENCES

[1] Samuel Assefa, Danial Dervovic, Mahmoud Mahfouz, Robert Tillman, Prashant Reddy, and Manuela Veloso. Generating synthetic data in finance: opportunities, challenges and pitfalls. In *ACM International Conference on AI in Finance*, 2020.

[2] Junwei Bao, Duyu Tang, Nan Duan, Zhao Yan, Yuanhua Lv, Ming Zhou, and Tiejun Zhao. Table-to-text: Describing table region with natural language. In *AAAI*, 2018.

[3] Iz Beltagy, Kyle Lo, and Arman Cohan. SciBERT: A Pretrained Language Model for Scientific Text. In *EMNLP-IJCNLP*, 2019.

[4] Vadim Borisov, Tobias Leemann, Kathrin Seßler, Johannes Haug, Martin Pawelczyk, and Gjergji Kasneci. Deep neural networks and tabular data: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 2022.

[5] Vadim Borisov, Kathrin Seßler, Tobias Leemann, Martin Pawelczyk, and Gjergji Kasneci. Language models are realistic tabular data generators. In *ICLR*, 2023.

[6] Kaidi Cao, Colin Wei, Adrien Gaidon, Nikos Arechiga, and Tengyu Ma. Learning imbalanced datasets with label-distribution-aware margin loss. In *NeurIPS*, 2019.

[7] Nitesh Chawla, Kevin Bowyer, Lawrence Hall, and Philip Kegelmeyer. SMOTE: synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 2002.

[8] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *KDD*, 2016.

[9] Wenhu Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyou Zhou, and Yang Wang. TabFact : A Large-scale Dataset for Table-based Fact Verification. In *ICLR*, 2020.

[10] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. Class-balanced loss based on effective number of samples. In *CVPR*, 2019.

[11] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative Adversarial Nets. In *NeurIPS*, 2014.

[12] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on tabular data? *arXiv preprint arXiv:2207.08815*, 2022.

[13] Hui Han, Wen-Yuan Wang, and Bing-Huan Mao. Borderline-SMOTE: a new over-sampling method in imbalanced data sets learning. In *International Conference on Intelligent Computing*, 2005.

[14] Haibo He, Yang Bai, Edwardo Garcia, and Shutao Li. ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In *IJCNN*, 2008.

[15] Haibo He and Edwardo Garcia. Learning from imbalanced data. *IEEE Transactions on knowledge and data engineering*, 2009.

[16] Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. Tabllm: Few-shot classification of tabular data with large language models. In *AISTAT*, 2023.

[17] Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Eisenschlos. TaPas: Weakly Supervised Table Parsing via Pre-training. In *ACL*, 2020.

[18] Jayoung Kim, Jinsung Jeon, Jaehoon Lee, Jihyeon Hyeong, and Noseong Park. Oct-GAN: Neural ODE-based conditional tabular GANs. In *Proceedings of the Web Conference*, 2021.

[19] Diederik Kingma, Max Welling, et al. An introduction to variational autoencoders. *Foundations and Trends in Machine Learning*, 2019.

[20] Muhammad Ferjad Naeem, Seong Joon Oh, Youngjung Uh, Yunjey Choi, and Jaejun Yoo. Reliable fidelity and diversity metrics for generative models. In *ICML*, 2020.

[21] Dang Nguyen, Sunil Gupta, Kien Do, Thin Nguyen, and Svetha Venkatesh. Generating realistic tabular data with large language models. In *ICDM*, 2024.

[22] Dang Nguyen, Sunil Gupta, Santu Rana, Alistair Shilton, and Svetha Venkatesh. Fairness Improvement for Black-box Classifiers with Gaussian Process. *Information Sciences*, 2021.

[23] Noseong Park, Mahmoud Mohammadi, Kshitij Gorde, Sushil Jajodia, Hongkyu Park, and Youngmin Kim. Data Synthesis Based on Generative Adversarial Networks. *Proceedings of the VLDB Endowment*, 2018.

[24] Zhaozhi Qian, Bogdan-Constantin Cebere, and Mihaela van der Schaar. Synthcity: facilitating innovative use cases of synthetic data in different data modalities. *arXiv preprint arXiv:2301.07573*, 2023.

[25] Fatih Sağlam and Mehmet Ali Cengiz. A novel SMOTE-based resampling technique trough noise detection and the boosting procedure. *Expert Systems with Applications*, 2022.

[26] Nabeel Seedat, Nicolas Huynh, Boris van Breugel, and Mihaela van der Schaar. Curated LLM: Synergy of LLMs and Data Curation for tabular augmentation in ultra low-data regimes. In *ICML*, 2024.

[27] Ravid Shwartz-Ziv and Amitai Armon. Tabular data: Deep learning is not all you need. *Information Fusion*, 2022.

[28] Yuan Sui, Mengyu Zhou, Mingjie Zhou, Shi Han, and Dongmei Zhang. Table meets LLM: Can large language models understand structured table data? a benchmark and empirical study. In *International Conference on Web Search and Data Mining*, 2024.

[29] Nan Tang, Ju Fan, Fangyi Li, Jianhong Tu, Xiaoyong Du, Guoliang Li, Sam Madden, and Mourad Ouzzani. RPT: relational pre-trained transformer is almost all you need towards democratizing data preparation. *VLDB Endowment*, 2021.

[30] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. Modeling tabular data using Conditional GAN. In *NeurIPS*, 2019.

[31] June Yong Yang, Geondo Park, Joowon Kim, Hyeongwon Jang, and Eunho Yang. Language-interfaced tabular oversampling via progressive imputation and self-authentication. In *ICLR*, 2024.

[32] Tianping Zhang, Shaowen Wang, Shuicheng Yan, Jian Li, and Qian Liu. Generative table pre-training empowers models for tabular prediction. In *EMNLP*, 2023.